Technical Summary: Progressive-Hint Prompting Improves Reasoning in Large Language Models

Kai-Yu Lu

1 Research Problem and Motivation

This study investigates how to improve the reasoning ability of large language models (LLMs, Large Language Models) on multi-step mathematical reasoning tasks only through prompt design, without additional fine-tuning or model modification. The central problem is that existing prompting strategies such as Chain-of-Thought prompting (CoT, Chain-of-Thought) and self-consistency use carefully designed prompts or multiple sampled reasoning paths, but they do not fully leverage the intermediate answers generated by the model itself as structured hints to guide subsequent reasoning.

From the perspective of research motivation, several limitations in prior work are identified. First, CoT-based methods focus on eliciting intermediate reasoning steps by demonstrating step-by-step solutions in the prompt, yet once a single answer is produced, the model typically does not re-examine or refine that answer using its own previous outputs. This contradicts how human problem solvers often behave, since humans frequently double-check an answer and use the first attempt as a hint to reconsider the problem.

Second, self-consistency methods propose to sample multiple reasoning paths and then aggregate them by majority vote to obtain a more reliable final answer. Although this approach improves accuracy, it can be computationally expensive, since many sampled paths are required to obtain a stable majority. The method also treats those paths as independent samples rather than as a sequence of hints that can be reused to guide later reasoning.

Third, prior work on reasoning path extraction and prompt engineering has not systematically explored automatic sequential interaction between a user and an LLM where past answers are explicitly incorporated into future prompts as hints. Consequently, there is a gap between current prompting strategies and a more human-like process of iterative self-refinement based on previous solutions.

2 Related Work

2.1 Multi-Step Reasoning and Chain-of-Thought Prompting

Multi-step reasoning tasks require models to perform several intermediate logical or arithmetic operations before reaching the final answer. Earlier studies have shown that LLMs are particularly strong at in-context learning, where the model follows patterns given in few-shot prompts to solve new tasks of the same type. Multi-step reasoning is one of the areas where large models exhibit pronounced emergent abilities.

Chain-of-Thought prompting (CoT) demonstrates that including explicit step-by-step reasoning examples in the prompt can significantly improve the performance of LLMs on arithmetic and logical reasoning tasks. Instead of directly asking for an answer, the prompt provides example solutions written as a sequence of reasoning steps, and the model is instructed to generate similar intermediate steps before the final answer.

Least-to-Most prompting extends this idea by decomposing a complex problem into simpler sub-problems, solving them in sequence and then combining the partial solutions. Complex CoT further emphasizes the importance of selecting difficult and complex questions as exemplars in the prompt, on the hypothesis that these exemplars better expose the reasoning patterns that the model should emulate. Auto-CoT is then introduced to reduce human effort by automatically mining and organizing appropriate prompts.

These methods collectively demonstrate that prompt design alone can elicit strong reasoning capabilities from LLMs. However, they mainly focus on the quality and structure of the initial prompt and do not consider subsequent interactions that reuse model-generated answers as hints.

2.2 Self-Consistency and Reasoning Path Extraction

Another influential line of work is self-consistency. Instead of relying on a single deterministic reasoning path, self-consistency samples multiple reasoning paths by using a stochastic decoder and then aggregates the final answers by majority vote. This method often improves robustness and accuracy because incorrect reasoning paths are more likely to be outvoted by correct ones.

Vote-complexity strategies further refine self-consistency by ranking sampled paths according to their reasoning complexity and selecting those with higher complexity for voting. This approach aims to focus on richer and more informative reasoning trajectories, which can be particularly beneficial on complex tasks.

Beyond prompt-level methods, prior research on reasoning path extraction has proposed techniques such as constructing semantic graphs, retrieving reasoning paths from knowledge graphs, or using human-annotated reasoning steps for fine-tuning. These methods target explicit identification or supervision of reasoning structures, but they typically require additional models, external knowledge resources, or labeled reasoning paths.

Overall, previous work either focuses on better prompt design or on sampling and aggregating multiple reasoning paths. None of these approaches explicitly models a sequential, interactive process where each generated answer is treated as a hint that influences the next round of reasoning.

3 Dataset Construction

The study evaluates the proposed method on seven public benchmarks that focus on mathematical reasoning. No new dataset is constructed; instead, the work carefully selects existing datasets that collectively cover a range of arithmetic difficulty and problem formats.

3.1 Elementary Arithmetic Word Problem Datasets

AddSub is a dataset of arithmetic word problems that primarily involve addition and subtraction. Each problem is a short textual description of a real-life scenario, such as counting items or combining quantities, and the target is a single numerical answer.

MultiArith contains multi-step arithmetic word problems that require several operations, for example combining addition, subtraction, and sometimes multiplication. Compared with AddSub, problems in MultiArith usually require more reasoning steps.

SingleEQ (Single Equation) includes word problems that can be written as a single algebraic equation. The textual description leads to a compact mathematical formulation, and solving the equation yields the final numeric answer.

SVAMP (Structural VARiations of Math Word Problems) is a benchmark designed to test robustness to structural variations. Many problems in SVAMP are derived from earlier datasets by paraphrasing or altering surface structure while preserving the underlying arithmetic relation. This makes it a challenging testbed for models that might rely on shallow patterns rather than true reasoning.

3.2 Grade-School and Higher-Level Mathematical Benchmarks

GSM8K is a large collection of grade-school math word problems covering topics such as arithmetic, basic algebra, and simple geometry. Problems are written in natural language and target the reasoning ability expected of proficient students at the grade-school level. The dataset is widely used as a standard benchmark for CoT-based reasoning with LLMs.

AQuA (Algebra Question Answering) consists of multiple-choice algebra and quantitative reasoning problems. Each example provides a problem statement and several candidate options. The task is to choose the correct option letter. This format tests both reasoning ability and the robustness of decision making under distractor options.

MATH is a challenging dataset of math competition problems spanning different topics and difficulty levels, often requiring nontrivial reasoning chains. Compared with the other benchmarks, MATH contains problems that are longer, more abstract, and less templated, making it a demanding test of higher-level mathematical reasoning.

Across all datasets, the study focuses on test-time evaluation only. Preprocessing is minimal and mainly involves formatting each problem into the prompt templates associated with different prompting strategies.

4 Query Protocol and Task Definitions

4.1 Task Definitions

All datasets are treated as mathematical reasoning tasks. For AddSub, MultiArith, SingleEQ, SVAMP, GSM8K, and MATH, each instance is a free-form word problem with a single numerical ground-truth answer. The model must produce a final number, possibly after generating intermediate reasoning steps.

For AQuA, each instance provides several candidate options. The model is expected to output either the correct option symbol or the correct numerical result while adhering to the multiple-choice format.

The main evaluation metric across all datasets is **accuracy**, defined as the proportion of problems for which the model's final answer exactly matches the ground-truth answer after appropriate normalization of numeric formats. For multiple-choice questions, accuracy is measured as the percentage of problems in which the correct option is selected.

4.2 Baseline Query Protocols

Three baseline prompting strategies are used to obtain base answers.

Standard prompting presents the question directly and asks for an answer without explicitly requesting intermediate steps. This is the simplest protocol and serves as a baseline for models' raw reasoning ability given only the problem statement.

Chain-of-Thought prompting (CoT) augments the prompt with few-shot examples that demonstrate explicit step-by-step reasoning. The model is encouraged to follow the pattern of "reasoning steps followed by answer" and therefore to expose intermediate computations before producing the final result.

Complex CoT uses more complex and informative examples as demonstrations. These examples are selected to have richer reasoning structure and higher difficulty, which is believed to better elicit the latent reasoning ability of powerful LLMs.

For all three prompting strategies, the base prompts are derived from the original CoT and Complex CoT papers. The study maintains the prompt content and focuses on how the new method can be applied on top of these established protocols.

4.3 Self-Consistency and Sample Paths

Self-consistency is adopted as an additional protocol for some experiments. Under self-consistency, the model is queried multiple times with stochastic decoding, generating multiple reasoning paths, often referred to as **sample paths**. Each path contains a reasoning chain and a final answer. The final prediction is determined by majority vote over the answers across paths.

The number of sample paths is varied across values such as 5, 10, 20, and 40, following the original self-consistency work. A higher number of sample paths typically improves robustness but increases computational cost. The study examines how the proposed method influences both accuracy and effective cost in this multi-path setting.

4.4 Interaction Number

The study introduces **interaction number** to measure how many times an agent must interact with the LLM to obtain the final answer under Progressive-Hint Prompting. The interaction number equals one for the base answer, two for the first subsequent answer, and so on. When a stopping criterion is satisfied, the interaction process terminates, and the final answer is declared. Interaction number thus captures the trade-off between the benefits of iterative refinement and the additional query cost.

5 Modeling Approach

5.1 Overview of Progressive-Hint Prompting

The central method in this study is Progressive-Hint Prompting (PHP, Progressive-Hint Prompting). PHP treats previous answers of the model as explicit hints and incorporates them into subsequent prompts to gradually guide the model toward a stable and correct answer.

The method is conceptually divided into two stages:

- **Stage 1: Base answer and base prompt.** Given a question, the model is first queried using a base prompt, which can be Standard, CoT, or Complex CoT. The model produces a base answer that serves as the starting point of the interaction.
- **Stage 2: Subsequent answers with Progressive-Hint Prompting.** The question is then re-asked together with a specially designed PHP prompt that explicitly lists hints derived from previous answers. The model generates a new answer conditioned on both the original question and the hint list. This process is repeated until the answer becomes stable.

The high-level intuition is that the model can "double-check" its own previous outputs. If earlier answers were incorrect but close to the correct value, the hint list provides useful guidance to refine the reasoning. If the hints already contain the correct answer, the model is encouraged to re-affirm it.

5.2 Hint Set Representation and Prompt Template

The PHP prompt template uses an explicit list of candidate answers as hints. The study describes these hints using symbols $A_1, \ldots, A_p$, where each A_i is a candidate answer that has been generated in a previous interaction or inserted as a designed hint. The hint set can be written as

$$H = \{A_1, A_2, \ldots, A_p\}. \tag{1}$$

In Equation (1), H denotes the set of answer hints. Each element A_i is a scalar answer candidate, typically a number representing a possible solution to the math problem. The index p represents the number of

distinct answer candidates currently considered. This formula does not introduce a new computation beyond the paper but formalizes the notation that already appears in the description of the prompt. Intuitively, H is the collection of answers that the model is told to consider "near" the correct answer. The role of this hint set is to provide a compact representation of previous answers so that they can be inserted into the PHP prompt.

The PHP prompt follows a two-sentence structure:

1. In the **question part**, a phrase of the form "The answer is near to $A_1, \ldots, A_p$" is appended after the original problem statement. This informs the model that plausible answers are close to the listed candidates.

2. In the **answer part**, the initial sentence states "We know the Answer Hints: $A_1, \ldots, A_p$. With the Answer Hints: $A_1, \ldots, A_p$, we will answer the question." This explicitly reminds the model of the hint values and instructs it to use them when generating its reasoning and final answer.

In realistic usage, the hint set may include a correct answer, incorrect answers, or a mixture of both. The method's design principle explicitly accounts for these possibilities. When hints coincide with the correct answer, the prompt encourages the model to reproduce and justify that answer. When hints are incorrect, the model is expected to "jump out" of these values and search for a better answer while still using them as reference points.

5.3 Progressive Interaction and Stopping Criterion

Progressive-Hint Prompting defines three interaction steps:

1. Generate a base answer using a base prompt (Standard, CoT, or Complex CoT).

2. Construct the hint set from previous answers and re-ask the question together with the PHP prompt to obtain a subsequent answer.

3. Repeat the second step, updating the hint set with answers collected so far, until the answer becomes stable.

The stopping criterion is defined as follows: the interaction is terminated when two consecutive answers are identical. Intuitively, once the model has generated the same answer twice in succession under different hint contexts, the system interprets this stability as evidence that the model has converged to a confident solution. This criterion enforces a finite interaction number and limits the additional computational cost of PHP.

The approach yields three main PHP variants:

- **PHP-Standard**: the base prompt is Standard, and subsequent prompts use the PHP structure derived from Standard.

- **PHP-CoT**: the base prompt is CoT, and subsequent prompts use PHP-CoT, which extends CoT with the hint phrases.

- **PHP-Complex CoT**: the base prompt is Complex CoT, and subsequent prompts use PHP-Complex CoT.

5.4 Design Principles for Hint Quality

The study emphasizes two key scenarios for hint quality:

1. **Hints equal to the correct answer.** In this case, the system verifies that the model can retain and justify the correct value when it is provided as a hint. Stability of answers under this scenario indicates that the prompt design does not disturb correct reasoning.

2. **Hints different from the correct answer.** Here, the system tests whether the model can escape incorrect values even when they are explicitly presented as hints. Effective performance in this scenario indicates that PHP helps the model reflect on and correct its own mistakes.

Experiments later confirm that including both correct and incorrect hints is beneficial. Correct hints encourage convergence to the right answer, while incorrect hints encourage exploration of alternative solutions.

5.5 Merge and Non-Merge Variants

To test the importance of separating base and subsequent prompts, the study introduces merge variants:

- **CoT-Merge** and **Complex CoT-Merge** combine the base prompt and PHP prompt into a single prompt used for both the base answer and subsequent answers.

In contrast, the non-merge versions use a pure CoT or Complex CoT prompt for the base answer and a dedicated PHP-CoT or PHP-Complex CoT prompt for subsequent answers. Experimental results show that non-merge PHP tends to outperform merge-based variants when prompts are more powerful, indicating that separating initial reasoning and progressive refinement is beneficial.

5.6 Combination with Self-Consistency

Progressive-Hint Prompting is designed to be orthogonal to self-consistency. The method can be integrated with self-consistency in two ways:

- **PHP on top of CoT or Complex CoT with self-consistency**: multiple reasoning paths are sampled, each path uses PHP-based interaction, and the final answers are aggregated by majority vote.

- **PHP to reduce required sample paths**: by enhancing each individual reasoning path through progressive hints, PHP can achieve high accuracy with fewer sample paths, reducing the computational cost of self-consistency.

This orthogonality is important because it shows that PHP does not compete with existing methods but can be combined with them to obtain further gains.

6 Empirical Results

6.1 Experimental Setup

The study evaluates PHP on the seven datasets using four LLMs accessed via the OpenAI API: text-davinci-002, text-davinci-003, GPT-3.5-Turbo, and GPT-4. Text-davinci-002 is reported as being fine-tuned with supervised instruction tuning, while text-davinci-003 is further fine-tuned with reinforcement learning. This difference in training leads to higher reasoning capability for text-davinci-003.

Model and Prompt	Dataset	Base Accuracy (%)	With PHP (%)
text-davinci-003 + Complex CoT	AddSub	86.3	88.1
text-davinci-003 + Complex CoT	MultiArith	94.8	95.0
text-davinci-003 + Complex CoT	SingleEQ	91.5	94.0
text-davinci-003 + Complex CoT	SVAMP	77.4	80.0
text-davinci-003 + Complex CoT	GSM8K	67.0	71.6
text-davinci-003 + Complex CoT	AQuA	48.8	50.0
GPT-4 + Complex CoT	SVAMP	89.1	91.9
GPT-4 + Complex CoT	GSM8K	92.0	95.5
GPT-4 + Complex CoT	AQuA	76.4	79.9
GPT-4 + Complex CoT	MATH	50.3	53.9

Table 1: Selected accuracy improvements obtained by Progressive-Hint Prompting on top of Complex CoT for text-davinci-003 and GPT-4.

All main results use greedy decoding, meaning that at each generation step the model selects the token with the highest probability (temperature equal to zero). This makes the behavior deterministic for a given prompt and highlights the effect of PHP on reasoning rather than on sampling noise.

6.2 Accuracy Improvements with Greedy Decoding

Table 1 summarizes key improvements when applying PHP on top of Complex CoT with text-davinci-003 and when combining PHP with GPT-4.

For text-davinci-003 with Complex CoT, PHP yields consistent improvements on all six datasets listed, with average accuracy increasing by more than two percentage points. For example, on GSM8K, accuracy increases from 67.0% to 71.6%, and on SVAMP it increases from 77.4% to 80.0%. These gains are substantial given that the baseline already uses a strong Complex CoT prompt.

For GPT-4, combining PHP with Complex CoT achieves state-of-the-art performance on several benchmarks: 91.9% on SVAMP, 95.5% on GSM8K, 79.9% on AQuA, and 53.9% on MATH. Compared with the corresponding base values, these represent absolute improvements of several percentage points and demonstrate that PHP continues to be effective even with very powerful models and prompts.

6.3 Effect of Model and Prompt Strength

The study reports that PHP tends to be more effective when applied to more powerful models and more powerful prompts.

Regarding model strength, text-davinci-003 generally benefits more from PHP than text-davinci-002 under the same prompting conditions. For example, PHP-Complex CoT brings an average gain of about 3.6% for text-davinci-002 and about 4.6% for text-davinci-003 across selected datasets. This pattern is consistent with the intuition that a stronger model can better interpret and utilize hints.

Regarding prompt strength, PHP improves performance only modestly when used with Standard prompts but provides larger gains when used with CoT and especially with Complex CoT prompts. This suggests that PHP complements the richer reasoning structures elicited by more powerful prompts.

6.4 Interaction Number and Efficiency

The interaction number analysis reveals two main trends:

- For a fixed prompt, the interaction number tends to be lower for stronger models. For example, text-davinci-003 often requires fewer iterations than text-davinci-002 to reach a stable answer when using the same PHP prompt. This is because stronger models more frequently produce a correct answer at an earlier step, leading to earlier convergence.
- For a fixed model, the interaction number generally increases with prompt strength. When moving from Standard through CoT to Complex CoT, the model can leverage hints to escape incorrect answers more effectively, but this may require additional iterations to settle on a stable answer. This represents a trade-off between achieving higher accuracy and incurring more interactions.

Despite the additional queries, the interaction number remains relatively low, and the overall cost is acceptable given the observed gains in accuracy.

6.5 Impact of Hint Quality

Experiments on hint quality show that replacing a weaker base prompt with a stronger one can greatly enhance PHP performance, especially for PHP-Standard. For instance, using the Standard prompt as the base for PHP-Standard yields only 16.0% accuracy on GSM8K, whereas using CoT as the base raises it to 50.2%, and using Complex CoT raises it further to 60.3%. This demonstrates that the quality of the hint list, determined by the base prompt's ability to generate good candidate answers, strongly affects the final performance.

Conversely, if a strong base prompt such as Complex CoT is replaced by a weaker one such as Standard when constructing PHP-Complex CoT, performance can drop. On GSM8K, replacing Complex CoT with Standard as the base prompt reduces accuracy from 71.6% to 65.5%. This confirms that strong hints are crucial for maximizing the benefits of PHP.

6.6 Ablation Study on Prompt Structure

An ablation study investigates two key sentences in the PHP answer template:

- P1: "We know the Answer Hints $A_1, \ldots, A_p$."
- P2: "With the Answer Hints $A_1, \ldots, A_p$, we will answer the question."

For CoT-based prompts, adding P1 and P2 yields modest improvements, whereas for Complex CoT-based prompts, the benefits are more pronounced. For example, Complex CoT on SVAMP improves from 78.0% to 80.0% after adding these sentences, and on GSM8K from 68.3% to 71.6%. This indicates that explicitly reminding the model of hints and instructing it to use them is particularly valuable when the underlying reasoning capacity is already strong.

The study also compares merge and non-merge variants. Complex CoT-Merge, which uses a single merged prompt for both base and subsequent answers, performs worse than non-merge PHP-Complex CoT when prompts are powerful. This suggests that separating base reasoning from progressive refinement is important for fully exploiting hints.

6.7 Correct vs. Incorrect Hints

Another ablation examines whether correct hints, incorrect hints, or both are necessary. For both CoT and Complex CoT, including either only correct hints or only incorrect hints generally improves performance over not using PHP at all. However, the best performance is usually achieved when both correct and incorrect hints are present.

Intuitively, correct hints help the model lock onto the right answer, while incorrect hints encourage the model to explore and discard misleading values. The combination of both types of hints allows PHP to both reinforce correct reasoning and correct earlier mistakes.

6.8 Performance with Self-Consistency

When combined with self-consistency, PHP further improves performance and can reduce the effective cost. For example, CoT with self-consistency achieves around 96.5% accuracy on MultiArith at certain sample path counts, while CoT with PHP and self-consistency can push accuracy higher using comparable or fewer sample paths. The abstract reports that on GSM8K, the proposed method with text-davinci-003 obtains a 4.2% improvement over Complex CoT with greedy decoding and reduces the number of required sample paths by approximately 46.17% under self-consistency.

These results show that PHP not only improves accuracy when used alone but also complements self-consistency by making each sampled reasoning path more reliable, which in turn reduces the number of paths needed for high-confidence voting.

7 Summary

This study proposes Progressive-Hint Prompting as a new prompting framework that uses previously generated answers as explicit hints to guide subsequent reasoning in large language models. The method is designed to be orthogonal to existing techniques such as Chain-of-Thought prompting and self-consistency and can be combined with them to achieve further gains.

In terms of research problem and motivation, the study addresses a clear gap: prior methods focus on prompt design and multi-path sampling but do not leverage model outputs as structured hints in a sequential interaction process. Progressive-Hint Prompting fills this gap by formalizing a pipeline that resembles human double-checking behavior.

The core methodological contribution lies in the two-stage interaction mechanism together with an explicit hint list and a carefully designed prompt template that repeatedly reminds the model of hints and instructs it to use them. The method is implemented in several variants, including PHP-Standard, PHP-CoT, and PHP-Complex CoT, and is further analyzed through merge and non-merge designs as well as ablation studies on sentence components and hint quality.

Empirical results on seven mathematical reasoning benchmarks and four strong LLMs demonstrate that PHP consistently improves accuracy, particularly when used with more powerful models and prompts. It reaches state-of-the-art performance in combination with GPT-4 on several benchmarks. The analyses on interaction number and self-consistency show that PHP also allows efficient reasoning by reducing the required number of sample paths to reach high performance.

Regarding limitations, the study focuses mainly on arithmetic and mathematical word problems. The generality of PHP for other forms of reasoning, such as commonsense reasoning, multi-hop question answering, or code generation, remains to be fully validated. Moreover, the design of hints and the interaction protocol is still largely handcrafted. Future work could explore automatic hint generation, adaptive stopping criteria, integration with external verifiers, or applications to broader task families.

Overall, the main takeaways are as follows:

1. Reusing previous answers as explicit hints in subsequent prompts provides a simple but effective mechanism for iterative reasoning improvement in large language models.

2. Progressive-Hint Prompting is complementary to existing techniques such as Chain-of-Thought prompting and self-consistency and can be combined with them to achieve state-of-the-art results on challenging math benchmarks.

3. Hint quality, prompt strength, and model power jointly determine the effectiveness of PHP, and careful design of the interaction protocol, including explicit hint sentences and non-merge prompts, is crucial for obtaining consistent gains.